VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Komponenta výukového serveru MAV - Kalkulus komunikujících systémů

Component of Learning Server for Modeling and Verification - Calculus of Communicating Systems

Bc. Jakub Koběrský

Diplomová práce

Vedoucí práce: Ing. Martin Kot, Ph.D.

Ostrava, 2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Č.j.VSB/25/140009
vsboee25040DC3

Zadání diplomové práce

Student: **Bc. Jakub Koběrský**

Studijní program: N0613A140034 Informatika

Téma: Komponenta výukového serveru MAV - Kalkulus komunikujících systémů
Component of Learning Server for Modeling and Verification - Calculus of Communicating Systems

Jazyk vypracování: čeština

Zásady pro vypracování:

V rámci diplomových a bakalářských prací vzniká výukový server pro předměty teoretické informatiky. Jedná se o sadu dynamických webových stránek umožňujících studentům pochopení různých typů úloh a problémů tím, že si mohou zadat na stránce libovolné zadání a zobrazí se jim řešení včetně postupu. Cílem této práce je vytvořit komponentu pro výuku vybraných problémů z oblasti formální verifikace systémů.

1. Nastudujte kalkulus komunikujících systémů (CCS), strukturální operační sémantiku (SOS) a ohodnocené přechodové systémy (LTS).
2. Vytvořte dynamické webové stránky umožňující uživateli následující:
a) Zadávat CCS výrazy - program zkontroluje syntaxi, zobrazí syntaktický strom výrazu.
b) Pro 2 CCS výrazy X,Y a daný symbol s tvořit důkaz podle pravidel SOS, zda existuje přechod z X do Y přes s. Tento důkaz bude mít možnost uživatel konstruovat interaktivně (bez nápovědy programu nebo s nápovědou, která pravidla je možné aktuálně použít).
c) Pro zadaný CCS výraz zobrazit odpovídající LTS (nebo jeho část, je-li LTS nekonečný).
d) Mít možnost simulovat běh jednoho CCS i několika paralelně běžících a vzájemně komunikujících CCS.
3. Vytvořte i ukázkové vstupy tak, aby uživatel mohl vše vyzkoušet i bez zadávání vlastních vstupů.

Seznam doporučené odborné literatury:

[1] Luca Aceto, Anna Ingólfsdóttir, Kim G. Larsen and Jiří Srba: Reactive Systems: Modelling, Specification and Verification. Cambridge University Press, August 2007.
[2] Christel Baier, Joost-Pieter Katoen: Principles of Model Checking, The MIT Press, 2008

17. listopadu 2172/15
708 00 Ostrava-Poruba
Česká republika
spojovatelka: +420 597 321 111
epodatelna: epodatelna@vsb.cz
ID datové schránky: d3kj88v
IČ: 61989100
DIČ: CZ61989100
email: studijni.fei@vsb.cz
www.fei.vsb.cz

Formální náležitosti a rozsah diplomové práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí diplomové práce: **Ing. Martin Kot, Ph.D.**

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: prof. RNDr. Václav Snášel, CSc.

V IS EDISON zadáno: 13.10.2025 09:58:20

Abstrakt

TODO

Klíčová slova

diplomová práce; TODO

Abstract

TODO

Keywords

master thesis; TODO

Poděkování

TODO

Obsah

Seznam použitých symbolů a zkratek **8**

Seznam obrázků **9**

Seznam tabulek **10**

1 Úvod **11**

2 Reaktivní systémy **12**

2.1 Rozdíl mezi klasickými a reaktivními systémy 12
2.2 Ohodnocené přechodové systémy 13
2.3 Kalkulus komunikujících systémů 14
2.4 Strukturální operační sémantika 16
2.5 Ekvivalence chování a logické vlastnosti 18
2.6 Rozšíření a další alternativy 20

3 Analýza požadavků **22**

3.1 Účel aplikace 22
3.2 Analýza existujících řešení 22
3.3 Funkční požadavky 23
3.4 Vedlejší požadavky 25

4 Použité technologie **27**

5 Návrh řešení **28**

5.1 Případy užití 28

6 Implementace **29**

7 Závěr **30**

Literatura **31**

Přílohy **31**

A Struktura přílohy **32**

Seznam použitých zkratek a symbolů

CCS	–	Calculus of Communicating Systems, kalkulus komunikujících systémů
SOS	–	Structural Operational Semantics, strukturální operační sémantika
LTS	–	Labelled Transition System, ohodnocený přechodový systém
AST	–	Abstract Syntax Tree, abstraktní syntaktický strom
HML	–	Hennessy-Milner Logic
CSP	–	Communicating Sequential Processes
ACP	–	Algebra of Communicating Processes
MaV	–	Modelování a verifikace
CAAL	–	Algebra of Communicating Processes
CWB	–	Edinburgh Concurrency Workbench
UX	–	User experience, Uživatelská přívětivost
UI/UX	–	User interface / User experience, Uživatelské rozhraní / Uživatelská přívětivost
PWA	–	Progressive web app, Progresivní webová aplikace

Seznam obrázků

2.1 LTS graf paralelní kompozice $P = a.0 \mid \overline{a}.0$ 16
2.2 LTS graf vynucené synchronizace $Q = (a.0 \mid \overline{a}.0) \setminus \{a\}$ 16

Seznam tabulek

Kapitola 1

Úvod

Kapitola 2

Reaktivní systémy

Tradiční pohled na počítačové programy je často zjednodušován na představu černé skříňky, která transformuje zadaný vstup na odpovídající výstup. Takovýto pohled na systém odpovídá popisu algoritmického problému. V praxi se však setkáváme s celou řadou počítačových systémů a aplikací, pro které tento zjednodušený pohled na jednorázovou transformaci není možný [1]. Typickými příklady jsou operační systémy, řídicí jednotky v automobilech, webové servery či běžné bankomaty a nápojové automaty. Tyto systémy, na rozdíl od tradičních algoritmů, nejsou navrženy tak, aby po zpracování dat svůj výpočet ukončily. Jejich primárním účelem je naopak průběžná a nepřetržitá interakce s okolním prostředím.

Tímto prostředím může být uživatel, další softwarové procesy, nebo přímo fyzický svět, ze kterého systém data čte pomocí senzorů. U reaktivních systémů se obecně předpokládá, že mohou teoreticky běžet do nekonečna a že jsou neustále připraveny reagovat na nové události, signály či požadavky pocházející zvenčí. Z tohoto důvodu je nekonečný běh u reaktivních systémů, na rozdíl od klasických programů, zcela přirozený a z hlediska jejich funkce často nezbytný [2].

2.1 Rozdíl mezi klasickými a reaktivními systémy

Základní rozdíl mezi klasickými (transformačními) a reaktivními systémy spočívá v samotné definici jejich korektního chování. U klasických programů, jako jsou například kompilátory, se zaměřujeme primárně na to, aby program pro korektní vstup po konečném počtu kroků skončil a vrátil správný výsledek [3]. Tyto programy lze poměrně dobře modelovat jako matematickou funkci či relaci, která mapuje množinu povolených vstupů na množinu výstupů. Během samotného výpočtu většinou není možné s těmito programy jakkoli interagovat a tím měnit jejich stav. Důkaz korektnosti je zde poměrně jednoduchý, protože často stačí dokázat, že pro každý platný vstup existuje konečný počet kroků, po kterých program skončí a vygeneruje správný výstup [1].

Naproti tomu reaktivní systémy jsou chápány jako nepřetržitě běžící celky. Jelikož u nich nelze snadno definovat konečné stavy značící úspěšný konec výpočtu, nelze je jednoduše zredukovat na

funkci či relaci [1]. Správnost reaktivního systému se nehodnotí jen podle finálního výsledku, ale zejména podle průběhu samotného výpočtu a vzájemné komunikace v čase. Při jejich verifikaci nás proto zajímají, oproti klasickým systémům, zcela jiné vlastnosti. Patří mezi ně bezpečnostní vlastnosti (*safety properties*), které zaručují, že v systému nikdy nenastane nežádoucí stav, jako je například uváznutí (*deadlock*) či porušení vzájemného vyloučení. Dále zkoumáme vlastnosti živosti (*liveness properties*), jež garantují, že se systém nezastaví a dříve či později obslouží každý požadavek [2].

Dalším velkým rozdílem těchto systémů je přítomnost paralelismu, asynchronního chování a s tím spojeného nedeterminismu. Zatímco klasický sekvenční program se pro stejná vstupní data chová vždy deterministicky, reaktivní systémy se typicky skládají z několika nezávislých komponent. Tyto procesy běží souběžně, sdílejí prostředky a reagují na externí události, jejichž přesné načasování nelze předvídat. Právě kvůli tomuto zabudovanému nedeterminismu u nich běžné metody testování často selhávají. Programátor pro ně může vytvořit sadu automatizovaných testů, které pokryjí většinu běžných scénářů, ale to nám nijak nezaručuje, že je systém opravdu bez chyb. Chyby v návrhu, jakými jsou například souběhy (*race conditions*), se často projeví až při velmi specifickém načasování událostí, které se často obtížně odhaluje a replikuje.

Právě tato nepřetržitost, složitá interakce a nedeterminismus jsou hlavními důvody, proč se pro návrh, analýzu a výuku reaktivních systémů používají specializované formální nástroje. Patří mezi ně zejména ohodnocené přechodové systémy (LTS), kterým se detailněji věnuje sekce 2.2, a procesní algebry, kam spadá i kalkulus komunikujících systémů (CCS), popsaný v sekci 2.3, jenž tvoří hlavní náplň praktické části této práce.

2.2 Ohodnocené přechodové systémy

Ohodnocené přechodové systémy (anglicky *Labelled Transition Systems*, zkráceně LTS) představují jeden ze základních formálních modelů pro popis chování reaktivních systémů. Díky této abstrakci můžeme modelovat celý systém jednoduše jako množinu stavů a možných přechodů mezi nimi. Každý z těchto přechodů je navíc ohodnocen určitou akcí, která daný krok popisuje. Tyto akce mohou reprezentovat události, ke kterým v systému dochází, jako je například přijetí vstupu od uživatele, odeslání zprávy po síti, nebo jen interní krok výpočtu. Velkou výhodou tohoto modelu je to, že jej můžeme intuitivně vizualizovat pomocí orientovaných grafů, kde uzly představují jednotlivé stavy systému a hrany značí přechody označené příslušnými akcemi [1].

Výhodou LTS je zejména jeho flexibilita a schopnost zachytit komplexní jevy. Snadno v něm můžeme modelovat jak běžné sekvenční chování, tak i nedeterminismus, kdy z jednoho stavu vede více přechodů se stejnou akcí do různých cílových stavů. Podobně lze pomocí LTS také odhalit stavy uváznutí, ze kterých už nevede žádná další cesta. Díky těmto vlastnostem můžeme přesně definovat a zkoumat vlastnosti systémů, včetně toho, za jakých podmínek považujeme chování dvou systémů za ekvivalentní, například pomocí silné a slabé bisimulace, popsané v sekci 2.5.1.

2.2.1 Formální definice LTS

Z matematického hlediska můžeme ohodnocený přechodový systém popsat jako trojici, případně čtveřici, pokud počítáme explicitně i s počátečním stavem [3]. Ohodnocený přechodový systém je definován jako trojice $(Proc, Act, \{\xrightarrow{\alpha} \mid \alpha \in Act\})$, kde:

- $Proc$ je množina stavů (procesů), která může být i nekonečná.
- Act je množina akcí (návěští neboli *labels*).
- $\xrightarrow{\alpha} \subseteq Proc \times Proc$ je binární přechodová relace pro každé $\alpha \in Act$.

Přechod $(s, s') \in \xrightarrow{\alpha}$ lze také chápat jako $(s, \alpha, s') \in \rightarrow$, obvykle se ale značí intuitivnějším zápisem $s \xrightarrow{\alpha} s'$. Tento zápis můžeme číst tak, že systém, který se momentálně nachází ve stavu s, může vykonat akci α a tím přejít do nového stavu s'.

V kontextu modelování pomocí kalkulu komunikujících systémů (CCS) se množina akcí Act rozděluje na dvě hlavní části. První část jsou viditelné akce, které reprezentují přímou komunikaci s okolím (vstupy a výstupy). Druhou částí je speciální neviditelná akce, značená řeckým písmenem τ (tau). Tato neviditelná akce reprezentuje buď interní krok systému, nebo výsledek synchronizace dvou běžících procesů, který zvenčí nelze pozorovat.

2.3 Kalkulus komunikujících systémů

Přestože ohodnocené přechodové systémy poskytují solidní matematický základ pro modelování, jejich přímé využití pro návrh rozsáhlých systémů naráží na limity. V praxi je modelování složitých systémů přímo pomocí LTS velmi zdlouhavé, nepřehledné a náchylné k chybám. Hlavní nevýhodou čistého LTS je nutnost popsat celý zkoumaný systém jako jeden velký a nedělitelný celek. Chybí nám tedy možnost při návrhu poskládat systém z menších nezávislých podsystémů [1].

Tento problém pomáhají řešit procesní algebry, z nichž jednou z nejvýznamnějších je Kalkulus komunikujících systémů (anglicky *Calculus of Communicating Systems*, zkráceně CCS). Tento formální jazyk, jehož autorem je držitel Turingovy ceny Robin Milner, si lze představit jako soubor výpočetních agentů komunikujících s okolím prostřednictvím svého definovaného rozhraní [4]. Hlavní výhoda CCS spočívá právě v jeho kompozičnosti. Umožňuje nám definovat chování malých komponent a pomocí sady operátorů je skládat do složitějších celků. V plné šíři své syntaxe (například i při využití operátorů restrikce či přejmenování uvnitř rekurze) je jazyk turingovsky úplný a dokáže tedy simulovat libovolný Turingův stroj. Jeho účel však nespočívá v provádění klasických algoritmických výpočtů, ale především v modelování a analýze chování interaktivních systémů [3].

2.3.1 Rozhraní a komunikace v CCS

Rozhraní libovolného procesu v CCS si můžeme představit jako sadu komunikačních portů (akcí), přes které proces interaguje se svým okolím, přičemž každý port slouží buď pro vstup, nebo výstup.

Komunikace v CCS je založena na principu vzájemné synchronizace dvou procesů (tzv. *handshake* komunikace). Abychom tuto komunikaci mohli modelovat, definuje CCS množinu akcí. Mějme $\mathcal{A}$ jako množinu jmen (reprezentujících například přijetí vstupní hodnoty na portu). Ke každému jménu $a \in \mathcal{A}$ existuje komplementární (opačné) jméno $\overline{a}$ (reprezentující odeslání hodnoty na daném portu). Množinu všech komplementárních jmen označíme jako $\overline{\mathcal{A}}$.

Vedle těchto běžných, navenek viditelných akcí zavádí CCS také již zmíněnou speciální tichou akci τ. Jelikož τ reprezentuje vnitřní krok, o kterém okolí neví, neexistuje k ní žádný komplement. Celková množina všech možných akcí Act v CCS je tedy sjednocením jmen, komplementů a tiché akce: $Act = \mathcal{A} \cup \overline{\mathcal{A}} \cup \{\tau\}$ [1].

2.3.2 Syntaxe CCS

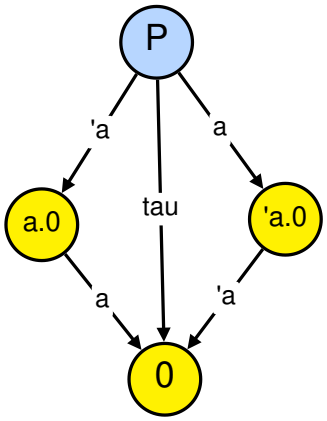
Syntaxe CCS je definována induktivně pomocí relativně malého množství operátorů, které přesto poskytují obrovskou vyjadřovací sílu [1]. Nechť $\alpha \in Act$ je libovolná akce, K je procesní konstanta (jméno procesu) a P je proces. Základní gramatiku jazyka lze poté definovat následovně:

$$P := K \mid \alpha.P \mid P_1 + P_2 \mid P_1|P_2 \mid P \setminus L \mid P[f] \mid 0$$

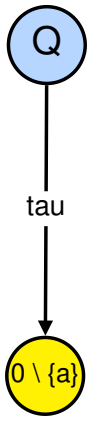
- **Procesní konstanta ($K \stackrel{\text{def}}{=} P$):** Umožňuje pojmenovávat procesy a odkazovat se na ně, díky čemuž dosahujeme rekurze.
- **Akční prefix ($\alpha.P$):** Proces, který nejdříve vykoná akci α a následně se začne chovat jako proces P, což si můžeme představit jako základní sekvenční operátor.
- **Nedeterministický výběr, suma ($P_1 + P_2$):** Představuje proces, který se může chovat buď jako proces P_1, nebo jako proces P_2. Výběr větve je závislý na tom, která akce je vybrána (ať už interně nebo okolím). Vykonáním akce z jedné větve se druhá alternativa trvale zahodí. Operátor lze také zapsat obecnou sumou $\sum_{i \in I} P_i$, kde I je libovolná množina indexů.
- **Paralelní kompozice ($P_1 \mid P_2$):** Umožňuje procesům P_1 a P_2 běžet souběžně. Procesy mohou vykonávat své akce nezávisle na sobě, nebo se mohou synchronizovat a společně vykonat interní akci τ, pokud je jeden připraven provést akci a a druhý její komplement $\overline{a}$. Tato kompozice je znázorněna na obrázku 2.1.
- **Restrikce ($P \setminus L$):** Kde $L \subseteq \mathcal{A}$. Tento operátor zakazuje (skrývá) vykonání akcí z množiny L a jejich komplementů. Běžně se využívá ve spojení s paralelní kompozicí k vynucení synchronizace mezi dvěma procesy tak, aby akce nebyly dostupné vnějšímu prostředí. Toto chování je znázorněno na obrázku 2.2, kde proces $P' = (a.0 \mid \overline{a}.0) \setminus \{a\}$ může vykonat pouze synchronizaci.
- **Přejmenování ($P[f]$):** Kde $f : Act \to Act$ je funkce s omezeními $f(\tau) = \tau$ a $f(\overline{a}) = \overline{f(a)}$. Tento operátor nám umožňuje znovu použít definované procesy v různých kontextech (např.

obecný automat $VM \stackrel{\text{def}}{=} coin.\overline{item}.VM$ můžeme přejmenovat na automat na kávu: $CM \stackrel{\text{def}}{=} VM[coffee/item]$) [1].

- **Prázdný proces (0 nebo *Nil*):** Značí stav, ve kterém proces nemůže vykonat žádnou další akci, a představuje tak korektní ukončení nebo uváznutí (*deadlock*).



Obrázek 2.1: LTS graf paralelní kompozice $P = a.0 \mid \overline{a}.0$



Obrázek 2.2: LTS graf vynucené synchronizace $Q = (a.0 \mid \overline{a}.0) \setminus \{a\}$

2.3.3 Vztah mezi CCS a LTS

I když nám syntaxe udává přesná pravidla pro zápis platných výrazů, k pochopení toho, co tyto výrazy skutečně znamenají a jak se chovají v čase, potřebujeme definovat jejich sémantiku. V případě CCS je sémantika formálně určena pomocí Strukturální operační sémantiky (SOS), která je popsána v následující kapitole 2.4. V principu ale platí, že každý syntakticky správný CCS výraz můžeme použít jako definici ohodnoceného přechodového systému. Díky tomuto propojení můžeme modely zapsané v CCS analyzovat pomocí algoritmů vytvořených primárně pro LTS a tyto modely graficky vizualizovat (např. s využitím nástroje implementovaného v rámci praktické části této práce). Při návrhu systémů tak můžeme využít modulárního přístupu procesní algebry provázaného s přechodovými systémy.

2.4 Strukturální operační sémantika

Jak již bylo řečeno v sekci 2.3.3, samotná syntaxe jazyka CCS pouze určuje, jak procesy správně zapisovat, neříká nám ale nic o tom, jaký význam tyto zápisy mají a jak se bude popsaný systém reálně chovat. Odvodit odpovídající stavy v modelu LTS je možné i intuitivně, tento postup je však velmi náchylný k chybám a u rozsáhlých systémů jej nelze spolehlivě automatizovat. Právě proto se k tomuto odvození využívá Strukturální operační sémantika (SOS), která definuje sadu

formálních odvozovacích pravidel [1]. Jejím autorem je Gordon Plotkin a poskytuje exaktní postup, jak z chování jednotlivých malých podkomponent krok za krokem logicky odvodit chování celého komplexního systému.

2.4.1 Odvozovací pravidla

Základním stavebním kamenem Strukturální operační sémantiky je systém odvozovacích pravidel (*inference rules*). Tato pravidla nám formálně říkají, za jakých podmínek může náš CCS proces provést určitou akci a přejít do nového stavu. Odvozovací pravidlo se standardně zapisuje ve tvaru zlomku [3]:

$$\text{PRAVIDLO} \quad \frac{\text{premisy}}{\text{závěr}} \quad \text{dodatečná podmínka}$$

Tento zápis čteme tak, že pokud platí všechny předpoklady (premisy) nad zlomkovou čarou a je i splněna případná dodatečná podmínka vedle zlomku, pak z toho logicky vyplývá platnost závěru pod čarou. Pravidlům bez premis říkáme axiomy, jejich závěr totiž platí automaticky za všech okolností.

Pro jazyk CCS máme k dispozici ucelenou sadu těchto pravidel pro každý z operátorů. Nechť $\alpha \in Act$ je libovolná akce (viditelná či neviditelná τ) a $a \in \mathcal{A}$ je viditelné jméno akce. Sémantiku operátorů pak můžeme definovat následovně:

- **Akční prefix (ACT):** Toto pravidlo je axiomem a tvoří úplný základ výpočtu, neboť říká, že proces $\alpha.P$ může provést akci α a následně pokračovat jako proces P.

$$\text{ACT} \quad \frac{}{\alpha.P \xrightarrow{\alpha} P}$$

- **Nedeterministický výběr (SUM):** Pravidla definují, že proces poskládaný ze součtu P_1+P_2 může provést buď krok procesu P_1, nebo krok procesu P_2. Provedením vybrané akce druhá větev zcela zaniká. Pravidlo lze také zapsat obecnou sumou.

$$\text{SUM}_1 \quad \frac{P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'} \qquad \text{SUM}_2 \quad \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}$$

- **Paralelní kompozice (COM):** Paralelní běh modelujeme pomocí tří pravidel. První dvě (COM_1 a COM_2) umožňují takzvané asynchronní prokládání (interleaving), kdy jeden z procesů vykoná svůj krok a druhý zůstává ve svém aktuálním stavu. Třetí pravidlo (COM_3) popisuje synchronizaci, ke které dochází, když jeden proces umí vyslat $\overline{a}$ a druhý přijmout a. Výsledkem této synchronizace je tichá akce τ.

$$\text{COM}_1 \quad \frac{P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q} \qquad \text{COM}_2 \quad \frac{Q \xrightarrow{\alpha} Q'}{P \mid Q \xrightarrow{\alpha} P \mid Q'}$$

$$\text{COM}_3 \quad \frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\overline{a}} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'}$$

- **Restrikce (RES):** Proces obalený restrikcí se může nadále chovat stejně jako proces původní, avšak s tím rozdílem, že má zakázáno provést jakoukoli akci z množiny zakázaných akcí L, včetně jejího komplementu.

$$\text{RES} \quad \frac{P \xrightarrow{\alpha} P'}{P \setminus L \xrightarrow{\alpha} P' \setminus L} \quad \alpha, \overline{\alpha} \notin L$$

- **Přejmenování (REL):** Pokud vykoná proces nějakou akci α, proces obalený funkcí přejmenování vykoná akci $f(\alpha)$.

$$\text{REL} \quad \frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f(\alpha)} P'[f]}$$

- **Procesní konstanta (CON):** Toto pravidlo umožňuje nahradit jméno definovaného procesu za jeho vnitřní tělo (proces P) a vykonat krok, který toto tělo nabízí.

$$\text{CON} \quad \frac{P \xrightarrow{\alpha} P'}{K \xrightarrow{\alpha} P'} \quad K \stackrel{\text{def}}{=} P$$

2.4.2 Důkazové stromy (Derivační stromy)

Pomocí výše zmíněných SOS pravidel jsme schopni pro jakýkoli zadaný CCS výraz formálně dokázat, zda je z něj možné provést přechod do jiného stavu přes danou akci [1]. Tento proces dokazování probíhá iterativním aplikováním pravidel, většinou „odspodu nahoru“, čímž postupně budujeme strukturu zvanou odvozovací či důkazový strom (*derivation tree*). Důkazový strom je také možné sestrojit opačným směrem, kdy začínáme axiomem a aplikujeme pravidla do té doby, než dosáhneme požadovaného výrazu.

Kořenem takového stromu je tvrzení o přechodu, jehož existenci chceme ověřit, a jednotlivá zanoření a větvení představují pravidla, která jsme v daném kroku použili. Aby byl důkaz považován za platný, musí listy stromu tvořit výhradně axiomy (většinou se jedná o pravidlo ACT), u kterých již není třeba nic dalšího dokazovat. Pokud se nám pro testovaný přechod podaří zkonstruovat validní konečný strom zakončený platnými axiomy, včetně splnění všech dodatečných podmínek u zlomků, tak dostaneme matematický důkaz, že přechod skutečně existuje.

2.5 Ekvivalence chování a logické vlastnosti

Při navrhování systémů a tvoření modelů velmi často narazíme na potřebu porovnat dva nezávislé procesy a dokázat, že dělají „to samé“. Zatímco u běžných sekvenčních programů bychom pouze

srovnali výstupní data pro stejné vstupy, v prostředí neustále interagujících procesů toto neplatí. Nástrojem pro posuzování takové shody u reaktivních systémů je zkoumání takzvané ekvivalence chování a dokazování vlastností pomocí modálních logik [1].

2.5.1 Ekvivalence chování

Klíčovým konceptem pro určení ekvivalence v LTS a CCS je bisimulace. Její hlavní myšlenka spočívá v tom, že systémy považujeme za ekvivalentní ve chvíli, kdy dokážou vzájemně napodobovat své přechody do nekonečna. Základní a nejpřísnější formou je silná bisimulace (*strong bisimulation*), značená operátorem $\sim$. Dva procesy jsou silně bisimulární právě tehdy, když na každý krok prvního procesu umí druhý proces odpovědět naprosto identickým krokem pod stejnou akcí, a to včetně skrytých tichých akcí τ. Stavy, do kterých se oba procesy přesunou, musí toto pravidlo opět obousměrně splňovat. Silnou bisimulaci lze formálně dokázat například přes hru útočníka a obránce. Pro reálné systémy je tento způsob často příliš striktní.

Pro nasazení v reálných aplikacích se spíše používá slabá bisimulace (*weak bisimulation*), značená symbolem $\approx$. Při ní nás zajímá pouze viditelné chování (vstupy a výstupy), zatímco interní výpočty můžeme ignorovat [1]. Slabá bisimulace systémům dává větší volnost a umožňuje jim před nebo po provedení viditelné akce vykonat libovolný počet neviditelných τ přechodů. Lépe odráží perspektivu vnějšího pozorovatele, protože abstrahuje od vnitřního fungování komponent.

2.5.2 Hennessy-Milner logika

Kromě ověřování ekvivalence dvou celých systémů (*Equivalence Checking*) využíváme ve formální verifikaci také techniku ověřování modelů (*Model Checking*). Při ní testujeme, jestli konkrétní systém splňuje námi požadovanou vlastnost. Touto vlastností může být například podmínka, že se automat po stisknutí tlačítka dříve nebo později vrátí do výchozího stavu. Hlavním nástrojem pro formulaci těchto lokálních vlastností reaktivních systémů je Hennessy-Milner logika (HML).

Jedná se o modální logiku, která rozšiřuje klasickou výrokovou logiku o dva speciální operátory, kterými jsou možnost ($\langle a \rangle F$) a nutnost ($[a]F$) [3]:

- Formule $\langle a \rangle F$ je splněna ve stavu s, pokud z tohoto stavu existuje *alespoň jeden* přechod přes akci a do stavu, který splňuje formuli F.

- Formule $[a]F$ je splněna ve stavu s, pokud *všechny* možné přechody přes akci a vedou do stavů, které splňují formuli F. Tuto formuli triviálně splňuje i jakýkoliv stav, ze kterého pod akcí a žádný přechod vůbec nevede.

O dvou procesech, které mají konečné větvení (*image-finite* LTS), platí tvrzení, že jsou silně bisimulární právě tehdy, když splňují stejnou množinu formulí v HML [1].

2.6 Rozšíření a další alternativy

Kalkulus komunikujících systémů (CCS) byl jednou z prvních procesních algeber a položil teoretické základy verifikace. S postupným rozvojem teoretické informatiky však vznikaly i další rozšíření nebo úplně nové matematické formalismy, které na CCS navazují.

2.6.1 CCS s předáváním hodnot

Základní verze CCS, jak byla představena v předchozích kapitolách, modeluje komunikaci pouze jako vzájemné odeslání signálu na určitém kanálu (tzv. handshake synchronizace), aniž by při této události docházelo k výměně jakýchkoli stavových dat. V reálném světě si však procesy mezi sebou potřebují vyměňovat konkrétní datové hodnoty, jako jsou například celá čísla, stavová data či textové řetězce. Pro tyto účely existuje rozšíření zvané CCS s předáváním hodnot (*Value-passing CCS*), ve kterém se komunikace obohacuje o parametry [3].

Přijetí hodnoty x na portu in se modeluje jako $in(x).P$, zatímco odeslání konkrétní hodnoty v (či výrazu) na portu out se zapisuje jako $\overline{out}(v).P$. Důležitým bodem je, že pokud v modelu pracujeme pouze s omezeným konečným počtem hodnot datového typu, nepřináší nám toto rozšíření z teoretického hlediska žádnou novou expresivitu. Každý takový proces lze totiž přeložit do standardního CCS tak, že se příjem hodnot nahradí velkou sadou nedeterministických výběrů pro každou možnou hodnotu datového typu a odeslání by se mapovalo na staticky definovaný specifický název akce (např. odeslání čísla 2 jako akce $\overline{out(2)}$).

2.6.2 Alternativy pro reaktivní systémy

Paralelně s vývojem CCS vznikala i procesní algebra CSP (*Communicating Sequential Processes*), jejímž autorem je sir C. A. R. Hoare [5]. Zatímco v CCS je komunikace založena primárně na komplementárních zprávách synchronizovaných přes neviditelnou akci τ, CSP vnímá komunikaci jako sdílení událostí, do kterých se může synchronně zapojit i více paralelních procesů současně (tzv. *multi-way synchronization*). Abstrakce, kterou algebra CSP zavedla, se v softwarovém inženýrství ujala a výrazně ovlivnila návrh moderních programovacích jazyků. Příkladem je jazyk Go, který se z CSP přímo inspiroval při návrhu svých `goroutines` a komunikačních kanálů.

Dalším významným pohledem na modelování komunikujících systémů je algebraický přístup, reprezentovaný formalismem ACP (*Algebra of Communicating Processes*), který vytvořili Jan Bergstra a Jan Willem Klop [6]. Ten, na rozdíl od CCS, které zakládá svou sémantiku zespodu (*bottom-up*) induktivně pomocí Strukturální operační sémantiky (SOS) a důkazových stromů, popisuje chování systémů primárně axiomaticky shora (*top-down*). Chování systému a ekvivalence procesů jsou v ACP formálně odvozovány převážně dedukcí ze sady definovaných rovnic tvořících danou algebru.

Snad nejvýznamnějším pokročilým formalismem, zavedeným autorem CCS Robinem Milnerem na začátku devadesátých let, je π-kalkul (*Pi-calculus*) [7]. Tento kalkul byl vyvinut jako odpověď na

nutnost modelovat takzvané mobilní procesy, tedy systémy, u kterých se může dynamicky za běhu měnit síťová a komunikační topologie. Zatímco v klasickém CCS jsou komunikační kanály (jména akcí) staticky svázané s architekturou systému, π-kalkul umožňuje předávat samotná jména kanálů jako komunikační data skrze jiné kanály (tzv. *link passing* či *name passing*). Tento mechanismus dynamického vytváření, delegování a sdílení nových referencí v čase z něj dělá ultimátní a vysoce expresivní nástroj pro modelování moderních distribuovaných internetových architektur, mikroservis a bezdrátových sítí s proměnlivým připojením klientů.

Kapitola 3

Analýza požadavků

Tato kapitola se věnuje analýze požadavků na vyvíjenou webovou aplikaci, jejímž cílem je podpora výuky vybraných problémů z oblasti formální verifikace systémů. V následujících sekcích je specifikován primární účel aplikace, zhodnocení existujících nástrojů a podrobná specifikace funkčních i nefunkčních požadavků definovaných na základě zadání.

3.1 Účel aplikace

Primárním účelem vyvíjené aplikace je sloužit jako interaktivní vizuální pomůcka pro studenty předmětu Modelování a verifikace (MaV), případně obdobných kurzů zaměřených na formální metody. Teorie formální verifikace a modelování konkurentních systémů pomocí kalkulu komunikujících systémů (CCS) může být pro studenty zpočátku složitější na pochopení, zejména pokud nemají k dispozici adekvátní nástroj pro vizualizaci chování těchto systémů.

Hlavním přínosem aplikace je provedení studenta prvními fázemi studia problematiky. Uživatel si může interaktivně vyzkoušet sestrojení ohodnoceného přechodového systému (LTS) na základě definovaných CCS výrazů, případně krok za krokem dokazovat platnost přechodů za užití pravidel strukturální operační sémantiky (SOS).

Jedním z hlavních cílů aplikace je tento vzdělávací proces usnadnit poskytnutím okamžité vizuální zpětné vazby. Aplikace umožní studentům experimentovat s vlastními procesy, vizualizovat jejich syntaktickou strukturu, krokovat jejich běh a především interaktivně budovat formální důkazy pomocí SOS pravidel. Díky tomu si mohou uživatelé lépe představit abstraktní principy synchronizace, nedeterminismu a komunikace mezi procesy.

3.2 Analýza existujících řešení

Oblast formální verifikace a analýzy systémů popsaných pomocí CCS již disponuje řadou aplikací. Většina z nich však byla navržena primárně pro výzkumné nebo průmyslové účely, nikoliv explicitně

jako pomůcky pro začátečníky. Mezi nejvýznamější programy v kontextu existujícího softwaru je vhodné zmínit následující aplikace.

3.2.1 Concurrency Workbench Aalborg (CAAL)

Jako nejvýznamnější moderní nástroj lze uvést CAAL (Concurrency Workbench Aalborg) [8]. Jedná se o komplexní webovou aplikaci umožňující specifikaci procesů v CCS, generování LTS, ověřování bisimulačních ekvivalencí a model checking pomocí modální logiky. Je to poměrně intuitivní nástroj, který se i během výuky předměnu používá, obsahuje však i určité limity. Hlavní nevýhodou je absence podpory pro dokazování přechodů pomocí jednotlivých SOS pravidel, což omezuje jeho využití v úvodních fázích výuky. Při generování LTS grafů navíc CAAL využívá algoritmy optimalizované spíše pro rozsáhlé stavové prostory, což má za následek horší přehlednost u menších výukových příkladů. Nástroj rovněž neposkytuje vizualizaci abstraktního syntaktického stromu (AST), která je pro pochopení sémantiky a priorit operátorů klíčová.

3.2.2 Edinburgh Concurrency Workbench (CWB)

Historicky důležitým nástrojem, který definoval standard pro automatizovanou analýzu CCS, je CWB [9]. Ačkoliv se jedná o spolehlivý software s vysokým výkonem, v dnešní době již není dále vyvíjen. Jeho zásadním nedostatkem pro účely moderní výuky je zastaralé uživatelské rozhraní založené výhradně na příkazové řádce. Neposkytuje žádnou formu interaktivní vizualizace grafů či syntaktických stromů, což jej činí pro podporu výuky těžko přístupným.

3.2.3 Zhodnocení a přínos nové aplikace

Z analýzy existujících řešení vyplývá, že hlavním nedostatkem je přílišná komplexita uživatelského rozhraní a absence nástrojů zaměřených čistě na pochopení základních principů (např. krokové budování SOS důkazů a vizualizace AST). Hlavním přínosem nového řešení je tak primárně zaměření na výuku. Aplikace nabídne validaci syntaxe v reálném čase, možnost vizuální nápovědy při volbě SOS pravidel a moderní, čisté prostředí. Stává se tak vhodným doplňkem (zejména pro úvodní přednášky a cvičení) k robustnějším nástrojům, nikoliv však jejich náhradou.

3.3 Funkční požadavky

Funkční požadavky přímo definují chování a možnosti vyvíjené aplikace. Vycházejí z oficiálního zadání diplomové práce a jsou rozšířeny o potřeby nezbytné pro reálné nasazení ve výuce:

1. **Zadávání a syntaktická analýza CCS výrazů**
    - Systém musí obsahovat textový editor pro zápis zdrojového kódu (procesních definic) v syntaxi jazyka CCS.

- Aplikace musí v reálném čase provádět syntaktickou analýzu vstupu a vizuálně upozorňovat uživatele na případné chyby (např. chybějící závorky, nepovolené znaky, nedefinované procesy).
- K zadanému a syntakticky korektnímu výrazu musí systém umožnit vygenerování a zobrazení syntaktického stromu (AST), reprezentujícího hierarchickou strukturu výrazu a prioritu operátorů.

2. **Interaktivní konstrukce odvozovacího stromu (SOS)**

   - Pro uživatelem zvolené procesy X a Y a akci α musí systém umožnit uživateli sestavit formální důkaz přechodu $X \xrightarrow{\alpha} Y$ na základě axiomů a odvozovacích pravidel strukturální operační sémantiky.
   - Tento proces musí probíhat interaktivně: uživatel postupně vybírá pravidla aplikovatelná na daný podvýraz, včetně jejich příslušných parametrů.
   - Aplikace musí nabízet dva režimy podpory výuky:
     (a) *Režim bez nápovědy:* Uživatel provádí volbu pravidel samostatně. V případě chybného kroku jej aplikace pouze upozorní, že dané pravidlo s příslušnými parametry nelze použít.
     (b) *Režim s nápovědou:* Systém na základě aktuálního stavu výrazu interaktivně filtruje a zvýrazňuje pouze ta SOS pravidla, která jsou syntakticky aplikovatelná, a omezuje výběr parametrů na validní možnosti.

3. **Generování a vizualizace ohodnoceného přechodového systému (LTS)**

   - Na základě zadaného CCS výrazu musí systém vygenerovat odpovídající stavový prostor a zobrazit jej jako orientovaný graf, kde vrcholy reprezentují stavy prostoru a orientované hrany jednotlivé akce.
   - Systém musí být odolný vůči zacyklení v případech, kdy je generovaný LTS nekonečný (např. vlivem neomezeného vytváření nových paralelních komponent nebo nezastavené rekurze). Generování musí být omezeno či přizpůsobeno tak, aby nedošlo k zamrznutí uživatelského rozhraní.

4. **Simulace běhu procesů**

   - Aplikace musí obsahovat interaktivní simulátor umožňující manuální krokování běhu procesu napříč vygenerovaným LTS grafem.
   - Uživatel musí mít možnost v každém stavu zvolit, kterou z dostupných akcí chce provést.
   - Simulátor musí umožňovat návrat o krok zpět, případně celou simulaci vrátit do výchozího stavu.

- Při analýze paralelních procesů (využívajících operátor $|$) simulátor vizuálně demonstruje nezávislé provádění akcí i interní komunikaci (handshake) přes komplementární porty (např. α a $\overline{\alpha}$), vedoucí ke vzniku tiché akce τ.

5. **Předdefinované ukázkové vstupy**

   - Systém musí obsahovat sadu předpřipravených příkladů demonstrujících použití jednotlivých operátorů CCS a typických situací ve formální verifikaci.
   - Tyto ukázky musí být možné načíst na jedno kliknutí bez nutnosti psaní kódu, což urychlí seznámení uživatele s aplikací.

3.4 Vedlejší požadavky

Vedlejší (nefunkční) požadavky definují vlastnosti systému, které nesouvisejí přímo s hlavní funkcionalitou verifikace, ale jsou klíčové pro celkovou uživatelskou přívětivost (UX), stabilitu a moderní standardy vývoje webových aplikací.

1. **Podpora offline režimu a PWA architektura:** Aplikace by měla být navržena jako progresivní webová aplikace (PWA). Logika výpočtů a zpracování CCS by mělo probíhat primárně na straně klienta, což umožní plnohodnotné využití nástroje i bez stabilního připojení k internetu. Ukládání rozpracovaných příkladů by mělo být řešeno pomocí lokálního úložiště prohlížeče, aby bylo možné uchovat rozpracovaný stav přímo v prohlížeči i bez přístupu k internetu.

2. **Lokalizace uživatelského rozhraní (i18n):** Aplikace by měla podporovat minimálně češtinu a angličtinu. Přepnutí jazyka musí proběhnout plynule bez nutnosti opětovného načtení celé stránky. Architektura aplikace by měla umožňovat snadné přidání dalších jazyků v budoucnosti.

3. **Export a vizualizace dat:** Aplikace musí umožnit export vygenerovaných syntaktických stromů a LTS grafů. Ideální je i podpora exportu do rastrových (PNG) i vektorových (SVG) formátů, aby je studenti mohli snadno začlenit do svých úkolů či prací. Vhodný je i export struktury do strojově čitelného formátu (např. JSON).

4. **Sdílení programů:** Systém musí poskytovat mechanismus pro snadné sdílení vytvořených procesů. Uživatel by měl mít možnost jednoduše zkopírovat kód nebo sdílet rozpracované řešení, což zefektivní komunikaci mezi uživateli.

5. **Přístupnost a responsivní design:** Přestože se předpokládá primární využití na desktopových zařízeních, rozhraní musí být responsivní a funkční i na mobilních telefonech a tabletech. Aplikace by měla být také přístupná podle zákonu o přístupnosti, platném od roku 2025. I

přesto, že se na tuto aplikaci zákon momentálně nevztahuje, měla by respkeptovat metodiku WCAG 2.2. [10]

6. **Integrovaná nápověda a dokumentace:** Součástí uživatelského rozhraní musí být kontextová nápověda (např. tooltipy vysvětlující jednotlivá SOS pravidla) a centrální dokumentace, která stručně objasní jak ovládání samotné aplikace, tak základní pojmy.

7. **Moderní UI/UX s využitím animací:** Vizuální stránka by měla působit čistě a moderně. Přechody stavů a úpravy grafů by měly být podpořeny plynulými animacemi. Tyto animace nemají plnit pouze estetickou funkci, ale také sloužit ke snížení kognitivní zátěže uživatele (např. plynulé zobrazení kroků při animaci průchodu grafem).

8. **Možnost uzamknutí programu:** Rozhraní by mělo umožnit uzamčení textového editoru, zadání SOS důkazů a dalších prvků aplikace. Toto uzamčení bude sloužit zejména jen jako ochrana proti nechtěným změnám, uživatel si ji může libovolně u programů aktivovat či deaktivovat.

9. **Strukturální redukce LTS grafu:** Systém by měl uživateli nabídnout možnost zjednodušit generované grafy a stromy, například automatickým odstraňováním zbytečných `Nil` procesů či aplikací základních pravidel strukturální kongruence.

Kapitola 4

Použité technologie

Kapitola 5

Návrh řešení

5.1 Případy užití

Kapitola 6

Implementace

Kapitola 7

Závěr

Literatura

1. ACETO, Luca; INGOLFSDOTTIR, Anna; LARSEN, Kim G; SRBA, Jiří. *Reactive systems: Modelling, specification and verification*. Cambridge University Press, 2007.
2. BAIER, Christel; KATOEN, Joost-Pieter. *Principles of model checking*. MIT press, 2008.
3. KOT, Martin. *Modelování a verifikace: Přednášky 1 až 5*. 2023.
4. MILNER, Robin. *Communication and concurrency*. Prentice-Hall, Inc., 1989.
5. GRAHAM, S; RIVEST EDITORS, R; HOARE, C. *Programming Teclmiques Communicating Sequential Processes* [online]. 1978. [cit. 2020-05-27]. Dostupné z: `https://www.cs.cmu.edu/~crary/819-f09/Hoare78.pdf`.
6. BERGSTRA, J.A.; KLOP, J.W. Algebra of communicating processes with abstraction. *Theoretical Computer Science* [online]. 1985, roč. 37 [cit. 2026-03-15]. Dostupné z DOI: `10.1016/0304-3975(85)90088-x`.
7. KOBAYASHI, Naoki; PIERCE, Benjamin C; TURNER, David N. Linearity and the pi-calculus. *ACM transactions on programming languages and systems* [online]. 1999-09, roč. 21, s. 914–947 [cit. 2024-06-02]. Dostupné z DOI: `10.1145/330249.330251`.
8. *CAAL* [online]. Cs.aau.dk, [b.r.]. [cit. 2026-03-15]. Dostupné z: `https://caal.cs.aau.dk/`.
9. *The Edinburgh Concurrency Workbench* [online]. Ed.ac.uk, [b.r.]. [cit. 2026-03-15]. Dostupné z: `https://homepages.inf.ed.ac.uk/perdita/cwb/`.
10. *10. Zákon o přístupnosti [Přístupné webové stránky]* [online]. Ipk.nkp.cz, 2019. [cit. 2026-03-15]. Dostupné z: `https://prirucky.ipk.nkp.cz/pristupnost/zakon_o_pristupnosti`.

Příloha A

Struktura přílohy